Baseball Transfer Manager

Documentación Técnica del Proyecto

Nombre del Proyecto	Baseball Transfer Manager
Versión	1.0.0
Lenguaje	Java
IDE	NetBeans
Base de Datos	MySQL
Autores	Surky Alexandra Báez Joshua Abreu Gilbert Vido Lucas Guerra Pedro Parra
Fecha de Lanzamiento	20 Abril 2026
Patrones de Diseño	MVC
Enlace al repositorio GitHub	https://github.com/surkyb/Baseball-Transfer-Manager

Descripción General del Proyecto

El Sistema de Mercado de Fichajes es una aplicación de escritorio desarrollada en Java que permite simular la gestión de transferencias de jugadores en un entorno deportivo profesional. Su objetivo es

ofrecer una plataforma organizada, eficiente y escalable para la administración de datos deportivos

1.1 Objetivo del Sistema

El sistema tiene como finalidad centralizar la información de jugadores, equipos y contratos, automatizar procesos clave del mercado de fichajes y facilitar la toma de decisiones estratégicas mediante datos actualizados.

1.2 Tecnologías Utilizadas

Tecnología	Versión / Detalle	Uso
Java	JDK 8 o superior	Lenguaje principal para el desarrollo del sistema
Swing / JFrame	Librerías estándar de Java	Desarrollo de la interfaz gráfica de usuario
MySQL	MySQL Server 5.7 o superior	Base de datos relacional para almacenamiento de información
POO	(Programación Orientada a Objetos)	Modelado de entidades y lógica del sistema
MVC	Modelo - Vista - Controlador	Patrones de Diseño
NetBeans	IDE de desarrollo	Entorno utilizado para desarrollar y ejecutar la aplicación

Arquitectura del Sistema

El sistema está organizado en una arquitectura por capas, lo que permite una mejor separación de responsabilidades, facilitando el mantenimiento, escalabilidad y organización del código.

2.1 Diagrama de Capas

Capa de Presentación (Vistas - Java Swing):
Interfaz gráfica que permite la interacción del usuario con el sistema. Incluye módulos como Login, Dashboard, gestión de jugadores, equipos, contratos y transferencias.

Capa de Negocio / DAO:
Encargada de la lógica de negocio y del acceso a los datos mediante interfaces y sus implementaciones. Gestiona las operaciones principales del sistema como transferencias, validaciones y reglas del negocio.

Capa de Datos (MySQL):
responsable del almacenamiento persistente de la información. Contiene las tablas relacionadas con las entidades del sistema como jugadores, equipos, contratos y transferencias.

Funcionalidades Principales

Función	Descripción
Login de usuario	Acceso seguro al sistema mediante autenticación.
Registro de equipos	Creación y gestión de equipos deportivos.
Gestión de jugadores	Registro, consulta y administración de jugadores.
Asignación de jugadores a equipos	Relación directa entre jugadores y equipos.
Transferencias entre equipos	Simulación de fichajes dentro del sistema.
Gestión de contratos	Control de inicio, duración y vencimiento.
Agentes libres automáticos	Los jugadores pasan a estado libre cuando su contrato expira.
Actualización de valor del jugador	El valor se ajusta dinámicamente según sus estadísticas.
Consulta y comparación	Visualización y análisis de jugadores y equipos.

3.1 Clase ConexiónDB

Ubicación: src/com/techbridge/btm

Métodos disponibles:

Método	Retorno	Descripción
conectar()	Connection	Crea y retorna una nueva conexión a MySQL
GetConexion()	Connection	Alias de conectar(), usada por los DAOs

Nota de seguridad importante:

Las credenciales (usuario y contraseña) están escritas directamente en el código fuente. Para entornos de producción se recomienda moverlas a un archivo de configuración externo (config.properties o variables de entorno) para evitar exponer datos sensibles.

Modelos (Entidades del Sistema)

Los modelos son clases POJO (Plain Old Java Object) que representan las entidades del negocio. Cada modelo tiene sus atributos privados con métodos getter y setter correspondientes.

4.1 Estado contrato

Paquete: Modelos | Archivo: Contrato.java

Atributo	Tipo	Descripción
estado	String	Estado actual del contrato

4.2 Jugador

Paquete: Modelos | Archivo: Jugador.java

Atributo	Tipo	Descripción
Id	int	Identificador único del jugador (PK)
nombre	String	Nombre completo del cliente
edad	int	Edad del jugador
posición	String	Posición del jugador
estadísticas valor de mercado estado(activo/ agente libre)	String double String	Estadísticas del jugador Valor del jugador en el mercado Estado del jugador en el mercado

4.3 Estadísticas

Paquete: Modelos | Archivo: Estadísticas.java

Atributo	Tipo	Descripción
Id	int	Identificador único de las estadísticas (clave primaria)
jugador_id	int	Identificador del jugador asociado (clave foránea)
promedioBateo	double	Promedio de bateo del jugador

hits	int	Cantidad de hits realizados
carreras	int	Número de carreras anotadas

homeRuns	int	Número de jonrones
juegosJugados	int	Cantidad de juegos disputados

4.4 Equipo

Paquete: Modelos | Archivo: Equipo.java

Atributo	Tipo	Descripción
Id	int	Identificador único del equipo (clave primaria)
nombre	String	Nombre del equipo
presupuesto	double	Presupuesto disponible del equipo para fichajes
plantilla (lista de jugadores)	List<Jugador> / int	Lista de jugadores pertenecientes al equipo

4.5 Contrato

Paquete: Modelos | Archivo: Contrato.java

Atributo	Tipo	Descripcion
id	int	Identificador único del contrato (clave primaria)
Jugador_id	int	Referencia al jugador (clave foránea)
equipo_id	int	Referencia al equipo (clave foránea)
fecha_inicio	Date / String	Fecha de inicio del contrato
fecha_fin	Date / String	Fecha de finalización del contrato
salario	double	Salario acordado para el jugador

4.6 Transferencia

Paquete: Modelos | Archivo: Transferencia.java

Atributo	Tipo	Descripción
Id	int	Identificador único de la transferencia (clave primaria)
Jugador	int	Referencia al jugador transferido (clave foránea)
Equipo_origen	int	Equipo desde el cual se transfiere el jugador (clave foránea)
Equipo_destino	int	Equipo que recibe al jugador (clave foránea)
Fecha	Date / String	Fecha en que se realiza la transferencia
valor	double	Valor económico de la transferencia

4.7 Usuario

Paquete: Modelos | Archivo: Usuario.java

Atributo	Tipo	Descripción
Id	int	Identificador único de la transferencia (clave primaria)
username	String	Nombre de usuario utilizado para iniciar sesión
password	String	Contraseña del usuario (se recomienda almacenar encriptada)
rol	String	Rol del usuario dentro del sistema (ej: administrador, usuario)
estado	String	Estado del usuario (activo o inactivo)

1. Capa de Acceso a Datos (DAO)

Cada entidad del sistema cuenta con una interfaz DAO y su implementación correspondiente. Todas las operaciones de base de datos utilizan PreparedStatement para prevenir inyección SQL.

1.1 Operaciones Comunes por Entidad

Entidad	Insertar	Actualizar	Eliminar	Buscar	Listar

Jugador	Si	Si	Si	Por ID	Si
Equipo	Si	Si	Si	Por ID	Si
Contrato	Si	Si	Si	Por ID	Si
Transferencia	Si	Si	Si	Por ID	Si

1.2 Tablas de Base de Datos

Tabla MySQL	Campos Principales
Jugadores	id, nombre, edad, posicion, estadisticas, valor_mercado, estado
Equipos	id, nombre, presupuesto
Contratos	id, jugador_id, equipo_id, fecha_inicio, fecha_fin, salario
Transferencias	id, jugador_id, equipo_origen, equipo_destino, fecha, valor

1.3 Lógica de Eliminación en Cascada

Para mantener la integridad referencial de la base de datos, el sistema implementa una lógica de eliminación en cascada a nivel de aplicación.

Cuando se elimina un **Jugador** o un **Equipo**, se eliminan primero los registros asociados para evitar errores de clave foránea.

Orden de eliminación:

Al eliminar un Jugador:

- Eliminar contratos asociados al jugador
- Eliminar transferencias relacionadas
- Eliminar el jugador

Al eliminar un Equipo:

- Eliminar contratos asociados al equipo
- Eliminar transferencias donde el equipo sea origen o destino
- Actualizar jugadores (si aplica) a estado *agente libre*
- Eliminar el equipo

2. Capa de Presentación (Vistas)

El sistema cuenta con formularios JFrame desarrollados con NetBeans GUI Builder. Cada vista tiene su archivo .java y su archivo .form correspondiente, permitiendo una interfaz gráfica estructurada y fácil de mantener.

Vistas del sistema:

Vista	Descripción y Funcionalidades
Login.java	Pantalla de inicio de sesión. Permite la autenticación del usuario para acceder al sistema
Dashboard.java	Panel principal que permite la navegación entre los diferentes módulos del sistema
Jugadores.java	CRUD completo de jugadores (registrar, editar, eliminar y listar jugadores)
Equipos.java	CRUD completo de equipos (crear, modificar, eliminar y visualizar equipos)
Contratos.java	Gestión de contratos entre jugadores y equipos (crear, actualizar y controlar vencimientos)
Transferencias.java	Gestión del mercado de fichajes, permitiendo transferir jugadores entre equipos
AgentesLibres.java *(opcional)*	Visualización de jugadores sin contrato disponibles para fichajes
Estadisticas.java *(opcional)*	Consulta y análisis del rendimiento de jugadores

3. Configuración y Despliegue

3.1 Requisitos del Sistema

Componente	Requisito
Java JDK	Versión 21 o superior
MySQL Server	Versión 8.x o superior
Sistema Operativo	Windows 10/11 (desarrollo en Windows)

NetBeans IDE	Versión 19+ (recomendado para editar formularios)

3.2 Configuración de la Base de Datos

Pasos para preparar la base de datos:

- Instalar MySQL Server y crear la base de datos: CREATE DATABASE baseball_transfer_manager;
- Ejecutar el script SQL para crear las tablas (jugadores, equipos, contratos, transferencias)
- Verificar que el usuario de base de datos (por ejemplo, `root`) tenga acceso local y permisos suficientes sobre la base de datos
- Configurar los parámetros de conexión en la clase `ConexionDB.java` (URL, usuario y contraseña)
- Para entornos de producción, se recomienda:
- Cambiar las credenciales por defecto
- Externalizar la configuración en archivos como `config.properties` o variables de entorno
- Restringir accesos a la base de datos para mayor seguridad

3.3 Pasos para Ejecutar el Proyecto

1. Clonar o descargar este repositorio
2. Abrir el proyecto en NetBeans (o cualquier IDE compatible con Java)
3. Configurar la conexión a la base de datos (si aplica)
4. Ejecutar la clase principal del sistema

4. Notas Técnicas y Recomendaciones

4.1 Observaciones del Código

- En algunas implementaciones DAO (por ejemplo, `JugadorDAOImpl` o `EquipoDAOImpl`), pueden existir métodos declarados en la interfaz que no están completamente implementados o utilizan implementaciones genéricas. Se recomienda validar que todos los métodos CRUD estén correctamente desarrollados.

- En el modelo **Jugador**, el manejo de atributos como `estadisticas` y `valorMercado` puede depender de valores tipo `String`. Se recomienda estandarizar estos campos a tipos numéricos cuando sea necesario para facilitar cálculos y validaciones.

- En algunas clases del sistema, pueden existir importaciones no utilizadas (por ejemplo, `java.sql.Date` o similares), lo cual genera advertencias de compilación. Se recomienda limpiar estos imports para mantener el código más limpio.

Es posible que algunos formularios (JFrame) no validen completamente los datos antes de enviarlos a la base de datos, lo que puede generar inconsistencias.

4.2 Mejoras Recomendadas

- **Seguridad de la base de datos:**
  Mover las credenciales de conexión a un archivo externo (`config.properties`) o utilizar variables de entorno para evitar exponer información sensible en el código.

- **Sistema de autenticación:**
  Implementar un sistema de login basado en una tabla de usuarios en la base de datos, con validación segura (hash de contraseñas).

- **Manejo de transacciones:**
  Implementar control de transacciones SQL (`commit` / `rollback`) en operaciones críticas como transferencias y eliminaciones en cascada.
- **Gestión de conexiones:**
  Utilizar `try-with-resources` en todas las clases DAO para garantizar el cierre adecuado de conexiones, `PreparedStatement` y `ResultSet`.
- **Validaciones en la interfaz:**
  Agregar validaciones en los formularios (JFrame) para:
- Campos obligatorios

- Tipos de datos correctos(edad, salario, valor)

- Formatos válidos

- **Optimización del sistema:**
  Separar aún más la lógica de negocio en la capa de servicios para evitar sobrecargar los DAOs.

- **Mejora del modelo de datos:**
  Normalizar la base de datos y asegurar el uso correcto de claves foráneas e índices para mejorar el rendimiento.